

## LAB7 – INF1316 – Sistemas Operacionais

### Paralelismo e Concorrência

#### 1. Identificação

Nome: Caio Faria Brito Martins Chaves

Matrícula: 2410162

Nome: Lucas Hufnagel Gromann de Araujo Goes

Matrícula: 2410845

#### 2. Objetivo

O objetivo deste laboratório é estudar, na prática, os conceitos de paralelismo e concorrência em sistemas operacionais, utilizando linguagem C em ambiente Unix/Linux.

O trabalho propõe a implementação de duas versões de um mesmo problema:

- uma versão com **processos irmãos**;
- uma versão com **threads**.

Em ambos os casos, foi utilizado um vetor de 10.000 posições inteiras, inicializado com o valor 5, sobre o qual 10 trabalhadores executam a operação:

$$\text{vetor}[i] = \text{vetor}[i] * 2 + 2$$

Além da implementação, o laboratório teve como finalidade:

- verificar automaticamente se todas as posições do vetor final possuem o mesmo valor;
- medir o tempo de execução utilizando `gettimeofday()`;
- desconsiderar o tempo de criação e de encerramento dos trabalhadores;
- comparar o comportamento de processos e threads;
- analisar o impacto do aumento do número de processos e do número de threads na eficiência do programa.

Esse objetivo está alinhado com a abordagem usada nos relatórios anteriores, que priorizam a observação prática de mecanismos do sistema operacional e a comparação entre comportamentos diferentes de execução concorrente.

#### 3. Estrutura do Programa

O trabalho foi dividido em dois programas.

##### 3.1 Programa com processos

O primeiro programa utiliza `fork()` para criar 10 processos trabalhadores irmãos.

De forma geral:

- o vetor é criado e inicializado pelo processo pai;
- os processos filhos são criados com `fork()`;
- cada filho percorre todas as 10.000 posições do seu vetor e aplica a operação:  
 $\text{vetor}[i] = \text{vetor}[i] * 2 + 2$ ;
- cada filho envia ao pai informações sobre o resultado obtido;
- o processo pai mede apenas a região de trabalho, sem contar a criação e o encerramento dos filhos;
- ao final, o programa verifica se todas as posições do vetor observado são iguais.

### 3.2 Programa com threads

O segundo programa utiliza a biblioteca `pthread`, conforme solicitado no enunciado, com compilação usando a opção `-pthread`.

De forma geral:

- o vetor é criado uma única vez no processo principal;
- são criadas 10 threads trabalhadoras com `pthread_create()`;
- todas as threads acessam o mesmo vetor global;
- cada thread percorre o vetor inteiro e aplica a mesma operação:  
 $\text{vetor}[i] = \text{vetor}[i] * 2 + 2$ ;
- o programa usa `pthread_join()` para aguardar o término das threads;
- o tempo é medido apenas durante a execução da região de trabalho.

Os slides anexados reforçam exatamente essa diferença: processos possuem espaços de memória separados, enquanto threads compartilham o mesmo espaço de endereçamento e os dados globais do processo. Eles também destacam que o acesso concorrente a dados globais precisa de exclusão mútua quando envolve mais de uma instrução de máquina.

## 4. Solução

### 4.1 Versão com processos

Na versão com processos, foram criados 10 processos irmãos. Cada processo executa a operação sobre todas as posições do vetor.

O ponto principal observado é que, após o `fork()`, cada filho passa a ter sua própria cópia do espaço de memória do processo pai. Isso significa que o vetor manipulado por um filho não é o mesmo vetor visível ao pai e aos demais filhos.

Na prática, cada processo filho transforma seu próprio vetor local. Como o vetor inicial começa com valor 5, após uma aplicação da fórmula:

$$5 * 2 + 2 = 12$$

cada filho passa a enxergar todas as posições com valor 12. Como cada processo trabalha em sua própria cópia, todas as posições dentro do vetor local daquele filho permanecem iguais.

Por outro lado, o vetor do processo pai não sofre alteração, pois ele não está em memória compartilhada. Assim, ao final da execução, o pai continua vendo o vetor original com valor 5 em todas as posições.

Portanto, nessa versão:

- dentro de cada filho, o vetor fica uniforme;
- no pai, o vetor permanece inalterado;
- não ocorre disputa direta entre os filhos sobre o mesmo vetor, porque cada processo trabalha em uma cópia própria.

Essa diferença entre processos e memória compartilhada já aparecia em um dos relatórios antigos, no qual a consistência dos dados dependia diretamente do tipo de memória utilizada e da forma de acesso concorrente.

## 4.2 Versão com threads

Na versão com threads, a situação muda completamente. As 10 threads executam dentro do mesmo processo e, por isso, compartilham o mesmo espaço de endereçamento e o mesmo vetor global.

Nesse caso, todas as threads acessam exatamente as mesmas posições do vetor ao mesmo tempo. Assim, surge o problema de concorrência sobre os dados compartilhados.

A operação:

$$\text{vetor}[i] = \text{vetor}[i] * 2 + 2$$

não é atômica. Ela envolve leitura, cálculo e escrita. Isso significa que duas ou mais threads podem ler o mesmo valor antigo de uma posição e sobrescrever umas às outras, gerando perda de atualização.

Dessa forma, mesmo que a lógica do cálculo esteja correta, a ausência de sincronização pode causar resultados incorretos ou diferentes do esperado. Esse tipo de situação caracteriza uma **condição de corrida**.

Os slides de threads deixam isso explícito ao afirmar que as threads compartilham os dados globais do programa e que, quando o acesso inclui mais de uma instrução, é necessário usar exclusão mútua.

Se as 10 atualizações fossem aplicadas de forma perfeitamente serial e sem perda, o valor esperado para cada posição seria obtido pela repetição da fórmula 10 vezes, partindo de 5.

Porém, como as threads executam concorrentemente sem proteção, podem ocorrer perdas de escrita.

Assim, na versão com threads, dois comportamentos podem ser observados:

- em algumas execuções, todas as posições ainda podem terminar com o mesmo valor;
- em outras, podem aparecer valores diferentes entre posições, ou um valor final menor que o esperado.

Mesmo quando todas as posições terminam iguais, isso não garante correção. Pode apenas significar que, naquela execução, o interleaving das threads não produziu uma diferença visível.

Esse comportamento não determinístico também aparece de forma semelhante no relatório anterior sobre concorrência com memória compartilhada, no qual os resultados variavam de uma execução para outra pela ausência de sincronização.

### **4.3 Medição do tempo com gettimeofday()**

A medição de tempo foi feita com a função gettimeofday(), conforme solicitado no enunciado.

Para evitar contar o tempo de criação e encerramento dos trabalhadores:

- no programa com processos, a contagem foi iniciada apenas quando todos os filhos já haviam sido criados e estavam prontos para começar o trabalho;
- no programa com threads, a contagem foi iniciada apenas após a criação das threads, no instante em que elas foram liberadas para executar;
- a contagem foi encerrada assim que o trabalho sobre o vetor terminou, antes da etapa final de encerramento completo da aplicação.

Essa forma de medir acompanha o padrão usado nos trabalhos anteriores da disciplina, em que gettimeofday() já era empregado para controlar intervalos de execução e observar o comportamento temporal dos programas.

### **4.4 Comparação entre processos e threads**

A comparação entre as duas versões mostra uma diferença importante de modelo de execução.

Na versão com processos:

- cada filho trabalha em sua própria cópia do vetor;
- não há disputa direta sobre o mesmo vetor;
- o pai não enxerga as alterações feitas pelos filhos;
- o custo de criação e troca de contexto tende a ser maior.

Na versão com threads:

- todas as threads trabalham sobre o mesmo vetor;
- existe compartilhamento real dos dados;
- há possibilidade de condição de corrida;
- o custo de gerenciamento tende a ser menor.

Os slides também destacam que a troca de contexto entre threads é mais leve do que a troca de contexto entre processos. Por isso, em geral, a versão com threads tende a apresentar melhor desempenho quando comparada à versão com processos, desde que o custo da sincronização não se torne dominante.

#### **4.5 Aumento do número de processos e threads**

Também foi proposto explorar o aumento do número de trabalhadores para observar a eficiência.

De maneira geral, o comportamento esperado é o seguinte:

- com poucos trabalhadores, pode haver ganho de desempenho devido ao paralelismo;
- à medida que o número de trabalhadores cresce, o custo de escalonamento e troca de contexto também cresce;
- em processos, esse custo tende a ser maior;
- em threads, o custo tende a ser menor, mas ainda assim existe disputa por CPU e por memória.

Além disso, no caso das threads, o aumento da quantidade de trabalhadoras sem sincronização pode aumentar a frequência das condições de corrida, tornando os resultados ainda mais imprevisíveis.

Assim, o aumento do número de processos ou threads não implica necessariamente aumento de eficiência. Depois de certo ponto, o overhead de gerenciamento passa a reduzir o ganho obtido pelo paralelismo.

### **5. Observações e Conclusões**

#### **Dificuldades**

As principais dificuldades encontradas neste laboratório foram:

- compreender com clareza a diferença entre paralelismo com processos e concorrência com threads;
- separar corretamente a região de medição de tempo, sem incluir criação e término dos trabalhadores;
- entender por que a versão com processos não altera o vetor observado pelo pai;
- analisar o comportamento não determinístico da versão com threads;

- identificar que a operação sobre o vetor não é atômica e, portanto, está sujeita a condição de corrida.

## **Resultados observados**

Os testes permitiram observar que:

- na versão com processos, cada filho modifica apenas sua própria cópia do vetor;
- o vetor do pai permanece inalterado;
- na versão com threads, o vetor é realmente compartilhado entre as trabalhadoras;
- sem sincronização, o resultado com threads pode variar entre execuções;
- a verificação automática de igualdade entre as posições permite detectar se o vetor final permaneceu uniforme;
- o uso de `gettimeofday()` foi adequado para comparar o custo de execução das duas abordagens.

## **Conclusão final**

O laboratório permitiu observar, de forma prática, uma diferença fundamental entre processos e threads em sistemas operacionais.

No caso dos processos, cada trabalhador executa de forma isolada em sua própria cópia de memória. Isso evita conflito direto sobre o vetor, mas impede o compartilhamento automático dos resultados com o processo pai.

No caso das threads, existe compartilhamento real da memória, o que torna a solução mais leve e, em geral, mais rápida. Em compensação, esse compartilhamento exige cuidado com sincronização. Sem mecanismos de exclusão mútua, o programa fica sujeito a condições de corrida e pode produzir resultados incorretos ou não determinísticos.

Portanto, o experimento mostra que threads tendem a ser mais eficientes do que processos do ponto de vista de custo de execução, mas essa eficiência vem acompanhada da necessidade de controle mais rigoroso sobre o acesso aos dados compartilhados. Essa conclusão está em linha com os conceitos apresentados nos slides sobre threads e também com a forma como os relatórios anteriores da disciplina analisam comportamento prático de concorrência e sincronização.